New Horizons
Centro de Capacitación

Práctica: Bandeja Multirrol con Acciones Condicionadas por Estado

Paso a paso — Selector de rol de usuario, botones contextuales de workflow y consumo de endpoints de transición

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 12 — Pantallas y Modales del Flujo de Aprobación End-to-End
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Implementar un conmutador de perfil/rol activo en el Shell (Solicitante, Revisor, Aprobador, Tesorería).
- Condicionar la visibilidad de botones de acción según el rol del usuario y el estado de la solicitud.
- Conectar el botón Enviar a POST /api/solicitudes/{id}/enviar (Borrador -> Enviada).
- Conectar el botón Iniciar Revisión a POST /api/solicitudes/{id}/revisar (Enviada -> EnRevision).
- Verificar que las transiciones de estado reflejen en tiempo real las reglas de la máquina de estados de C#.

2. Desarrollo paso a paso

Paso 1 — Crear el Servicio de Sesión de Usuario y Rol Activo

Construimos RolService en src/app/services/rol.service.ts para alternar roles organizativos:

src/app/services/rol.service.ts

```
import { Injectable, signal } from '@angular/core';

export enum RolUsuario {
  Solicitante = 'Solicitante',
  Revisor = 'Revisor',
  Aprobador = 'Aprobador',
  Tesoreria = 'Tesoreria'
}

export interface UsuarioActivo {
  id: number;
  nombre: string;
  email: string;
  rol: RolUsuario;
}

@Injectable({ providedIn: 'root' })
export class RolService {
  usuarioActual = signal<UsuarioActivo>({
    id: 1,
    nombre: 'Juan Pérez (Solicitante)',
    email: 'jperez@empresa.com',
    rol: RolUsuario.Solicitante
  });

  cambiarRol(nuevoRol: RolUsuario): void {
    let id = 1;
    let nombre = 'Juan Pérez (Solicitante)';
    if (nuevoRol === RolUsuario.Revisor) { id = 2; nombre = 'María Gómez (Revisora)'; }
    if (nuevoRol === RolUsuario.Aprobador) { id = 3; nombre = 'Carlos Soto (Gerente)'; }
    if (nuevoRol === RolUsuario.Tesoreria) { id = 4; nombre = 'Ana Ruiz (Tesorería)'; }

    this.usuarioActual.set({ id, nombre, email: 'usuario@empresa.com', rol: nuevoRol });
  }
}
```

Paso 2 — Agregar Endpoints de Transición a SolicitudHttpService

Ampliamos SolicitudHttpService en src/app/services/solicitud-http.service.ts con las operaciones de transición desarrolladas en la Sesión 6 de C#:

src/app/services/solicitud-http.service.ts

```
import { Injectable, inject, signal } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable, tap } from 'rxjs';
import { environment } from '../../environments/environment';
import { Solicitud, PagedResult, CrearSolicitudDto, ActualizarSolicitudDto, TimelineItem,
EstadoSolicitud } from '../models/solicitud.model';

@Injectable({ providedIn: 'root' })
export class SolicitudHttpService {
  private http = inject(HttpClient);
  private baseUrl = `${environment.apiUrl}/solicitudes`;

  solicitudes = signal<Solicitud[]>([]);

  obtenerPaginado(pagina: number = 1, tamano: number = 10, estado?: EstadoSolicitud):
Observable<PagedResult<Solicitud>> {
    let params = new HttpParams().set('pagina', pagina.toString()).set('tamanoPagina',
tamano.toString());
    if (estado !== undefined && estado !== null) {
      params = params.set('estado', estado.toString());
    }
    return this.http.get<PagedResult<Solicitud>>(this.baseUrl, { params }).pipe(
      tap(res => this.solicitudes.set(res.items))
    );
  }

  obtenerPorId(id: number): Observable<Solicitud> {
    return this.http.get<Solicitud>(`${this.baseUrl}/${id}`);
  }

  crear(dto: CrearSolicitudDto): Observable<Solicitud> {
    return this.http.post<Solicitud>(this.baseUrl, dto).pipe(
      tap(nueva => this.solicitudes.update(list => [nueva, ...list]))
    );
  }

  actualizar(id: number, dto: ActualizarSolicitudDto): Observable<void> {
    return this.http.put<void>(`${this.baseUrl}/${id}`, dto);
  }

  eliminar(id: number): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/${id}`).pipe(
      tap(() => this.solicitudes.update(list => list.filter(s => s.id !== id)))
    );
  }

  // --- TRANSICIONES DE FLUJO DE APROBACION ---
  enviar(id: number): Observable<Solicitud> {
    return this.http.post<Solicitud>(`${this.baseUrl}/${id}/enviar`, {});
  }

  iniciarRevision(id: number, revisorId: number = 2): Observable<Solicitud> {
    return this.http.post<Solicitud>(`${this.baseUrl}/${id}/revisar?revisorId=${revisorId}`, {});
  }

  aprobar(id: number, comentario: string, aprobadorId: number = 3): Observable<Solicitud> {
    return this.http.post<Solicitud>(`${this.baseUrl}/${id}/aprobar`, { comentario,
aprobadorId });
  }

  rechazar(id: number, motivo: string, responsableId: number = 3): Observable<Solicitud> {
    return this.http.post<Solicitud>(`${this.baseUrl}/${id}/rechazar`, { motivo,
responsableId });
  }

  pagar(id: number, medioPago: string, numeroOperacion: string, responsableId: number = 4):
Observable<Solicitud> {
    return this.http.post<Solicitud>(`${this.baseUrl}/${id}/pagar`, { medioPago, numeroOperacion,
responsableId });
  }

  obtenerTimeline(id: number): Observable<TimelineItem[]> {
    return this.http.get<TimelineItem[]>(`${this.baseUrl}/${id}/timeline`);
  }
}
```

Paso 3 — Actualizar el Shell con el Conmutador de Rol

En src/app/layout/shell/ incorporamos el selector de rol activo en sus tres archivos (.ts, .html y .css):

src/app/layout/shell/shell.component.ts

```
import { Component, signal, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatSidenavModule } from '@angular/material/sidenav';
import { MatListModule } from '@angular/material/list';
import { MatIconModule } from '@angular/material/icon';
import { MatButtonModule } from '@angular/material/button';
import { MatProgressBarModule } from '@angular/material/progress-bar';
import { MatSelectModule } from '@angular/material/select';
import { LoadingService } from '../../services/loading.service';
import { RolService, RolUsuario } from '../../services/rol.service';

@Component({
  selector: 'app-shell',
  standalone: true,
  imports: [
    CommonModule, FormsModule, RouterOutlet, RouterLink, RouterLinkActive,
    MatToolbarModule, MatSidenavModule, MatListModule, MatIconModule, MatButtonModule,
MatProgressBarModule, MatSelectModule
  ],
  templateUrl: './shell.component.html',
  styleUrl: './shell.component.css'
})
export class ShellComponent {
  loader = inject(LoadingService);
  rolService = inject(RolService);
  RolUsuario = RolUsuario;
  sidenavAbierto = signal<boolean>(true);

  cambiarPerfil(rol: RolUsuario): void {
    this.rolService.cambiarRol(rol);
  }
}
```

src/app/layout/shell/shell.component.html

```
<mat-toolbar color="primary" class="nh-toolbar">
  <button mat-icon-button (click)="sidenavAbierto.set(!sidenavAbierto())">
    <mat-icon>menu</mat-icon>
  </button>
  <span class="nh-logo-title">
    <mat-icon class="logo-icon">account_balance_wallet</mat-icon>
    SolicitudesApp — Flujo de Aprobación
  </span>
  <span class="spacer"></span>

  <!-- CONMUTADOR DE ROL PARA PRUEBAS DEL FLUJO -->
  <div class="role-switcher">
    <span class="role-label">Perfil Activo:</span>
    <mat-select [ngModel]="rolService.usuarioActual().rol"
(ngModelChange)="cambiarPerfil($event)" class="role-select">
      <mat-option [value]="RolUsuario.Solicitante">Juan Pérez (Solicitante)</mat-option>
      <mat-option [value]="RolUsuario.Revisor">María Gómez (Revisora)</mat-option>
      <mat-option [value]="RolUsuario.Aprobador">Carlos Soto (Gerente Aprobador)</mat-option>
      <mat-option [value]="RolUsuario.Tesoreria">Ana Ruiz (Tesorería)</mat-option>
    </mat-select>
  </div>
</mat-toolbar>
<mat-progress-bar *ngIf="loader.estaCargando()" mode="indeterminate" color="accent"
class="global-loader"></mat-progress-bar>

<mat-sidenav-container class="nh-sidenav-container">
  <mat-sidenav [opened]="sidenavAbierto()" mode="side" class="nh-sidenav">
    <mat-nav-list>
      <a mat-list-item routerLink="/dashboard" routerLinkActive="nh-item-active">
        <mat-icon matListItemIcon>dashboard</mat-icon>
        <span matListItemTitle>Dashboard</span>
      </a>
      <a mat-list-item routerLink="/solicitudes" routerLinkActive="nh-item-active">
        <mat-icon matListItemIcon>receipt_long</mat-icon>
        <span matListItemTitle>Bandeja Solicitudes</span>
      </a>
      <a mat-list-item routerLink="/solicitudes/nueva" routerLinkActive="nh-item-active">
        <mat-icon matListItemIcon>add_circle</mat-icon>
        <span matListItemTitle>Nueva Solicitud</span>
      </a>
    </mat-nav-list>
  </mat-sidenav>

  <mat-sidenav-content class="nh-content">
    <router-outlet></router-outlet>
  </mat-sidenav-content>
</mat-sidenav-container>
```

src/app/layout/shell/shell.component.css

```
.nh-toolbar { background: linear-gradient(90deg, #024b98 0%, #002d62 100%); color: white; }
.nh-logo-title { display: flex; align-items: center; gap: 8px; font-weight: 600; font-size:
1.05rem; }
.spacer { flex: 1 1 auto; }
.role-switcher { display: flex; align-items: center; gap: 8px; background:
rgba(255,255,255,0.15); padding: 4px 12px; border-radius: 8px; }
.role-label { font-size: 0.85rem; color: #e2e8f0; }
.role-select { color: white; width: 230px; }
.global-loader { position: absolute; top: 64px; left: 0; right: 0; z-index: 999; }
.nh-sidenav-container { height: calc(100vh - 64px); }
.nh-sidenav { width: 250px; background: #ffffff; border-right: 1px solid #e2e8f0; }
.nh-item-active { background: #eff6ff !important; color: #024b98 !important; font-weight: 600;
border-left: 4px solid #f57d30; }
.nh-content { padding: 24px; background: #f8fafc; overflow-y: auto; }
```

3. Retos adicionales

1. Añade un selector desplegable en el Header que permita cambiar de rol en caliente para demostración en vivo.
2. Implementa badges de conteo en el menú para mostrar cuántas solicitudes están esperando la acción del rol activo.
3. Bloquea la edición de la solicitud si su estado no es Borrador.
4. Muestra un mensaje de advertencia si un aprobador intenta aprobar una solicitud que excede su límite de firma.

4. Entregables

- Bandeja multirrol con botones contextuales habilitados según el estado del documento.
- Transición Borrador -> Enviada ejecutando exitosamente el endpoint de C#.
- Transición Enviada -> EnRevision ejecutando exitosamente el endpoint de C#.
- Actualización inmediata del estado visual y feedback con MatSnackBar.